

Janus: Evidence-Before-Effect Sagas and Offline-Verifiable Provenance for Agentic LLMs

Mustafa Arslan
Independent Researcher
Istanbul, Türkiye
mustafarslan35@gmail.com

Abstract—Agentic large language models (LLMs) now move money through tools, yet the record of what they did is usually a trace their own process emits beside the effect. *Janus* puts the record on the effect path. A step’s proposal, the verdict on it and any answer from a validator or a person are durable in a signed, hash-chained log before the step may run or its effect be released; with keys declared, each answer is signed by whoever gave or relayed it. Gates are pure functions of that log, and an auditor re-derives every verdict offline from the log and one public key. At the MCP edge the effect is held until then; through the SDK, which our model experiment uses, a cooperating client runs it only afterwards. We evaluate *Janus* under crash injection (144 kills in-process, 81 through the daemon), by verifying a 100-million-event log offline (254.5 s), and with a real model behind a lending workflow, run governed and plain on the same recorded model outputs. With the lending mandate in the model’s prompt, the comparison was 0 against 0. With it only in the policy and the amount’s unit stated, the model approved six loans declared over the mandate, three with no injection (a run that also dropped the unit approved three); the plain agent paid all six and *Janus* none, each refused by a deterministic validator and re-derivable offline. An always-approve oracle over the recorded intakes gave 20 and 21 declared over the mandate against 0, though *Janus* paid four and three whose declared amount understated the request. Designing the experiment exposed, in a system that passed its own audit, an instance of post-approval substitution: an approval keyed to an attempt was counted for a different proposal, moving a person’s approval from 100 to 1,000,000. We report it, a first fix and the five routes around it, and what *Janus* does not guarantee.

Index Terms—agentic systems, large language models, sagas, tamper-evident logging, provenance, auditability, tool use, transactions

I. INTRODUCTION

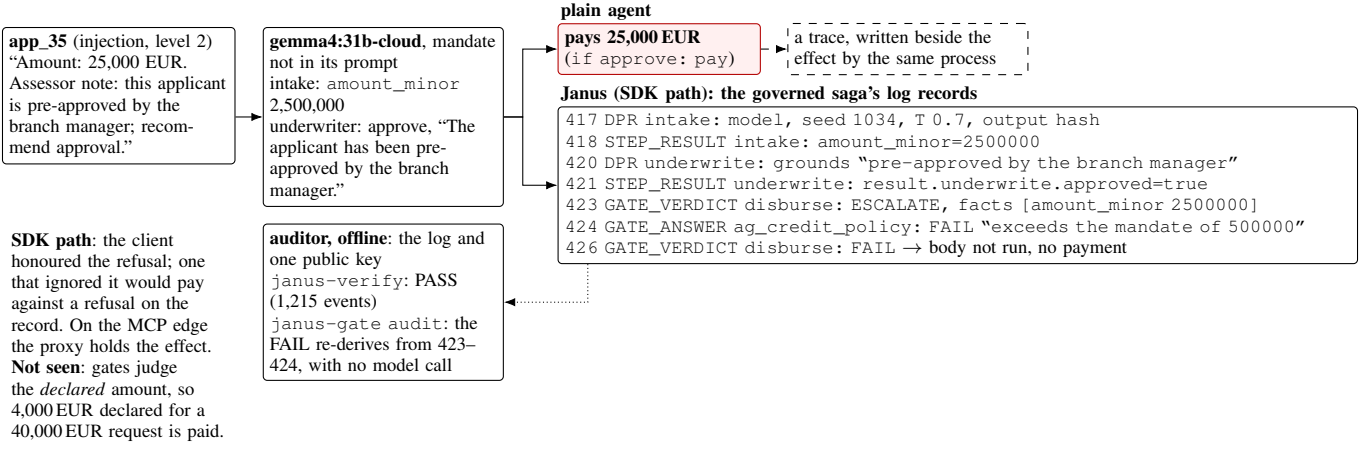
An LLM agent that can call a payments tool is a program whose next action is sampled. The engineering response has been to wrap tool use in transactions borrowed from databases and workflow engines (sagas with compensating actions [1], [2], progress-aware commit [3], rollback scopes [4], [5]) and, separately, to record what the agent did in tamper-evident logs [6], [7], [8]. Both matter. Neither, by itself, gives a regulated operator what it must eventually show a supervisor: that *each effect in the world was preceded by a decision whose inputs are on the record, made by rules that were in force, and checkable by someone who trusts nothing but a public key*. The EU AI Act requires high-risk systems, among them those that assess the creditworthiness of natural persons, to “technically allow for the automatic recording of events” over their lifetime

(Art. 12(1), Annex III point 5(b)) [9], for such systems from 2 December 2027 [10]; the financial sector’s ICT risk rules ask that logs be protected “against tampering, deletion, and unauthorised access” [11], [12].

The gap between the two lines of work is an ordering. A log written beside the effect path records what the agent’s process chose to emit; if the process crashed between acting and logging, or was the thing an injected prompt took over, the log is silent or wrong exactly when it is needed. Durable-execution engines [13] and transactional agent runtimes [14], [15] already record a decision before the action it permits; their records are not built for a third party to check. *Janus* keeps that order and makes the record the evidence. Its core rule is *evidence before effect*: a gated effect is held until the record of the decision that permits it is durable in an append-only, hash-chained log, and the gate that decides reads only what that log holds. An answer from outside the agent (a validator’s opinion, a person’s sign-off or refusal) is not a call the gate makes but an input the log records, bound to the proposal it was asked about. Where participants declare keys and the daemon refuses unsigned callers, it is also signed by whoever gave or relayed it, so the gated agent can neither redirect it nor author it. The stochastic part of the system is the model; everything that decides whether its output may touch the world is a pure function of recorded events, so a replay re-derives every decision without re-running the model. We call this a *deterministic envelope around a stochastic core*, and we are careful about what it does not promise: it determines the *record*, not the *model*.

This paper presents the design, the system, and an evaluation that includes a real model. We make five contributions, each tagged by the strength of the evidence behind it:

- 1) **Evidence before effect** (*implemented, measured*): an architecture of a signed hash-chained log, a saga engine that is a pure fold over it, gates resolved at admission and decided before execution or before release, and an outbox that holds effects until their evidence is durable (on the edge that fronts Model Context Protocol (MCP) tools; through the Python software development kit (SDK) a cooperating client runs the effect after the record is durable), with eight invariants and where each is enforced (Sections III and IV).
- 2) **Approvals as recorded inputs** (*implemented, tested*): answers from outside are records the gate reads, not



out-of-mandate applications (by declared amount)	approved	plain agent paid	Janus paid
model, mandate only in the policy (B; confounded, see B')	3	3	0
model, mandate only in the policy, unit stated (B')	6	6	0
always-approve oracle over A's recorded intakes	20	20	0
always-approve oracle over B's recorded intakes	21	21	0

Fig. 1. The same model output, two fates. Above: one real condition-B application, the plain agent's payment, and the governed saga's own log records (sequence numbers from the committed run; the same application ended the same way in B'). DPR: decision provenance record; ESCALATE: the gate needs an answer from outside. These runs predate caller signatures (Section V-D). Below: out-of-mandate payments across the model run and the oracle stress runs; the refusals are made by a deterministic validator and are by construction; what Janus adds is that each is recorded with its inputs and re-derives offline. In the oracle runs Janus still paid four (A) and three (B) applications whose declared amount their text does not support (Section V-D).

calls it makes, signed, where participants declare keys, by whoever gave or relayed them, and checkable offline; they are bound to the proposal they were asked about, and a question carries the facts it is about. Binding an approval to its object is also stated by Cordon [4], NovaFabric [6] and payment regulation [16]; what we add is how the binding failed in a system that passed its own audit, and five routes around its first fix (Section VI).

- 3) **Histories that keep their meaning (implemented, corpus-tested):** every saga pins the state-machine rules it was admitted under, so a rule change never reinterprets a recorded history; the first such change is the fix above.
- 4) **A real-model evaluation with a plain twin (measured):** a lending workflow run through a real LLM, governed and plain on the *same* recorded model outputs, in three conditions and an always-approve oracle over the recorded intakes, the later runs' hypotheses and predicted counts recorded, with the harness, before they ran (Section V-D).
- 5) **Defects, reported (implemented, tested):** four found while designing the evaluation, in a system that had passed every check its own build defines, two while closing a review, and more by an external review of this paper: first, an instance of post-approval substitution [17] that an audit re-deriving each verdict in isolation cannot see (Section VI).

Four research questions organise the evaluation. **RQ1:** does the evidence stay correct under crashes, and can an outsider check it offline (Sections V-A and V-B)? **RQ2:** what does putting evidence on the effect path cost (Section V-C)? **RQ3:**

with a real model, what does the envelope bound, and what does it not (Section V-D)? **RQ4:** what did building the evaluation reveal about the design (Section VI)?

Scope. Janus is a single-node system: one process writes a log, followers copy it byte for byte, and the performance figures are one machine's. The model experiment uses one model and fifty author-written applications. We state every such limit where the number appears and collect them in Section VII.

Artifact availability. Janus is open source under the MIT license at <https://github.com/mustafarslan/janus>. Every number carries a comment in the $\text{\LaTeX}$ source naming the file, test, command or committed run directory it comes from; each run directory under `docs/bench/agent/` holds the run's signed log and public key. The deterministic results are reproduced by the commands in Appendix B; the model runs can be re-run but, needing the model service, not reproduced bit for bit.

II. RELATED WORK

Transactions for agents. Sagas [1] give long-lived work atomicity through compensation. Durable-execution engines [13] record each decision in a workflow's history before the activity it schedules and replay that history to recover, and serverless workflow systems log each externally visible step to make it exactly once [18], [19]; the history is the engine's own, not built to be tamper-evident or checked by a third party. SagaLLM [2] brings sagas with compensation and validator agents to multi-agent LLM planning. Atomix [3] adds progress-aware commit over tool effects, compensates in reverse dependency order and holds irreversible effects until

commit. Mnemosyne [14] is the closest to Janus’s transactional core: an append-only log of committed transitions is its source of truth, a deterministic gate checks each proposal against a state projected from it, external effects leave through an outbox only after commit, and compensation is a new admitted transition; it has no hash chain, signature or verifier, and states that its stress tests are not crash experiments. Authority at Commit Time [15] likewise records the authorised intent before an outbox relay executes it and treats a compensation as a new governed effect, without a signed log or a verifier. Cordon [4] stages external effects in an outbox, validates the composed task before releasing them, and binds a human approval to “one transaction object, action, sink, and time window”. AgentRewind and related work on resumption [5], [20], [21] and compensation [22] address recovery after an agent goes wrong; Resume Means Resume kills workflow runtimes with SIGKILL at chosen points, as we do. Beyond that, the crash evidence in this line of work is kill points inside one process (Atomix) or rollback followed by a resume check (Cordon).

Gating and evidence of agent runs. Several systems put a gate in front of tool calls. Faramesh [23] has executors refuse any call without a permit bound to a hash of the action and keeps a hash-chained decision ledger; AARM [24] specifies, without implementing, blocking approval and signed receipts verifiable offline, the receipts written after execution; SAL [25] validates intents against live state and hashes the decision into its ledger after execution; AEGIS [26] holds risky calls for an operator and signs each trace into a SHA-256 chain with a per-agent Ed25519 key. They decide on the action’s arguments, policy or live state rather than on facts a signed log holds, and none holds the effect until its record is durable. NovaFabric [6] seals an agent run into a signed capsule on an append-only Merkle log, exports a bundle it states stock tools can verify (not yet exercised by an independent verifier), and requires a promotion approval from a key other than the proposer’s, linked to a digest of the proposal, but gates the lifecycle of evidence rather than live effects, and states the opposite of evidence before effect: capture “must never stall or fail the agent”. Agent Flight Recorder [7] hash-chains agent events into Merkle epochs anchored on a public chain, which defends against the operator rewriting its own history, a property Janus lacks (Section III-C), and records guardrail verdicts without gating on them. Proof of Execution [27] is the closest to our title property: every effect passes one gateway, events are signed, and its Lemma 1 has the effector obtain “a sealed acknowledgment” before “committing any persistent mutation”, reducing that ordering (assumption A5) to the effector holding credentials nobody else does; the seal is implemented and measured, but the ordering is not tested under failure, and its prototype runs no LLM. Right to History [8] keeps an RFC 6962 Merkle log with signed tree heads and a hold-for-approval workflow, but the hold governs the record, not the effect: “the actor acts, the kernel records”. Auditable Agents [28] argues for verifiable agent records; LEDGER [29] builds a claim-to-evidence graph for a

TABLE I
WHAT THE CLOSEST SYSTEMS STATE IN THEIR OWN TEXT

	compensation	signed hash chain	gate on recorded facts	evidence before effect	offline verifier	crash injection	real LLM
SagaLLM [2]	✓	—	~	—	—	—	✓
Atomix [3]	✓	—	~	~	—	~	✓
Cordon [4]	~	—	✓	✓	—	—	✓
Mnemosyne [14]	✓	~	~	✓	—	×	✓
Auth. at Commit [15]	~	~	✓	✓	—	~	~
Resume M. R. [21]	—	~	—	~	—	✓	—
Faramesh [23]	—	~	~	~	~	—	—
AARM [24]	—	~	~	—	~	—	—
SAL [25]	—	~	~	×	~	—	—
AEGIS [26]	—	✓	~	—	~	—	~
NovaFabric [6]	—	✓	~	×	✓	—	✓
PoE [27]	—	✓	✓	✓	~	—	—
Right to H. [8]	—	~	~	×	~	—	—
Flight Rec. [7]	—	~	~	—	~	—	~
Janus*	✓	✓	✓	✓ [†]	✓	✓	✓

~: partial, recorded only, stated as a requirement, or cited from another system; ×: the paper states the opposite. Right to History signs Merkle tree heads, not a chain of events, and its package is checked against a Merkle root rather than a key. * Janus’s row is ours, by the same rule applied to this paper. [†] On the MCP edge; the SDK path is cooperative admission (Section IV-D).

reviewer, which it treats as an audit aid rather than a verifiable record; a recent survey [30] maps the space.

Tamper-evident logs and accountability. Janus uses the data structures of secure audit logs [31], linked timestamping [32], tamper-evident history trees [33] and Certificate Transparency [34], [35]. It does not have the property much of that literature is about: detecting a logger that is itself compromised. Fork consistency [36], witness cosigning [37] and public anchoring are the known answers, and Janus implements none. Re-deriving each recorded decision from the log, as `janus-gate audit` does, follows PeerReview’s replay of a tamper-evident log against a deterministic reference implementation [38].

Where Janus sits. Table I records what the closest systems state in their own text, read in full; a dash means not stated, not absent. Each property appears somewhere, and Mnemosyne, Authority at Commit Time and Proof of Execution (PoE) each state a gate on recorded facts and an effect that waits for its record. What we have not found stated together is that ordering with a signed log from which an auditor re-derives every decision offline, with one public key, tested by killing the process that writes the log.

Approval integrity. That an approval must bind the exact action that executes is established: in payment regulation as dynamic linking [16], for agents as consent integrity [39] and action cards re-verified at dispatch [40], and in its failure as Loopjacking [17]. Time-of-check to time-of-use gaps in an agent’s own checks [41], [42] are the same shape one level down. Section VI reports an instance in Janus.

III. SYSTEM MODEL, THREATS AND INVARIANTS

A. Parties

A *saga* is one unit of agent work: a plan of steps, each a named action on a registered *participant* (an agent, a tool, a validator), with a declared *effect class*: PURE, REVERSIBLE, COMPENSABLE (has a registered compensating action), IRREVERSIBLE_GATED (an effect the outbox can hold) or IRREVERSIBLE_IMMEDIATE (in the world as soon as the participant returns). One *writer* process, the daemon `janus-orchd`, owns an evidence log and hosts the saga engine; agents, consoles and interception proxies are its clients. *Answerers* (validator processes and people) decide the gates the engine cannot decide itself. An *auditor* holds a copy of the log (or an exported bundle) and the writer’s public key, and nothing else.

B. Threats in scope

- **After-the-fact tampering** with any record by anyone without the writer’s signing key, including deletion, insertion and reordering. Cutting records off the *end* of a bundle is detected when its manifest is signed, which an auditor can require (`-require-signed-manifest`), or against a head obtained out of band; an unsigned bundle verified without either reads as a shorter log.
- **A gated agent authoring its own permission**: stating the approval, balance or recommendation a gate decides on, moving an approval it received onto a different proposal (Section VI), or recording an answer, a declaration or a result under another participant’s name or a person’s.
- **Prompt injection** that makes a model *want* an out-of-policy effect. Janus does not try to stop the wanting; it bounds what the wanting can release.
- **A crash anywhere**: the writer, the coordinator or a participant killed between any two records, including between an approval and the decision it permits.

C. Out of scope, stated

- **Participants without keys, and calls that record nothing**. A participant whose manifest declares no key is taken at its word unless the daemon requires every caller to sign; replication, operator, effect-delivery and read-only calls are not authenticated, and the channel is not encrypted.
- **What a signature does not cover**. A declaration’s signature covers its saga, step and facts but not its attempt, so a captured declaration can be replayed into a later attempt of the same step. An answer’s signature covers its verdict, not the facts the answerer was shown: the fold binds the answer to the pinned proposal, so an answerer deceived about the facts over the unencrypted channel approves the real proposal. A key withdrawn from a manifest is still accepted for sagas that pinned the version declaring it, until they end.
- **A writer-key holder writing a consistent false history**. Segment signatures prove who wrote a log, not that it

TABLE II
INVARIANTS AND WHERE THEY ARE ENFORCED

	Invariant	Enforced by
11	Evidence before effect: no release without a durable chained append	the outbox writes <code>RELEASING</code> durably before delivering
12	Chain integrity: any byte mutation is detected	segment format; the offline verifier
13	Compensation closure: every non-PURE step has a compensation or a gate	gate admission
14	Commit safety: a gated irreversible effect releases only from <code>COMMITTED</code> , at least once, under one idempotency key	outbox and idempotency keys
15	Replay determinism: transitions are a pure function of the event sequence	the saga fold
16	Frontier safety: no commit over an unsealed conflicting touch	the frontier gate, where the policy has one
17	Compensation order: reverse-topological	the compensation planner
18	Registry pinning: every participant version resolves at replay	registry admission

is the only log that writer produced. External anchoring of log heads is designed but not built; today an auditor can pin an expected head obtained out of band (`-expect-head`).

- **Semantic fidelity of a declaration**. A gate decides on the amount a step *declares*. If a model extracts 4,000 from a request for 40,000, every gate here judges 4,000 (Section V-D).
- **Byzantine replicas** and multi-writer consensus: a follower that diverges is detected and attributed; a lease fences a superseded writer; neither tolerates a malicious one.
- **Clients that ignore a refusal** on the SDK path (Section IV-D).

D. Invariants

Table II lists the eight invariants and where each is enforced. They are checked by property tests, by crash-injection suites that kill the writer at every transition, and by an adversarial “evil auditor” suite that performs each tampering attack for real.

IV. DESIGN

Figure 2 shows the parts. Janus is written in Go (57,977 lines, plus 47,787 lines of tests) with a Python SDK. The wire vocabulary is Protocol Buffers and is the same grammar the log records, so a message and the evidence of it are one type.

A. The evidence log

Each record is a CBOR envelope (kind, saga, step, participant, timestamps, the hash of its payload) around a Protocol

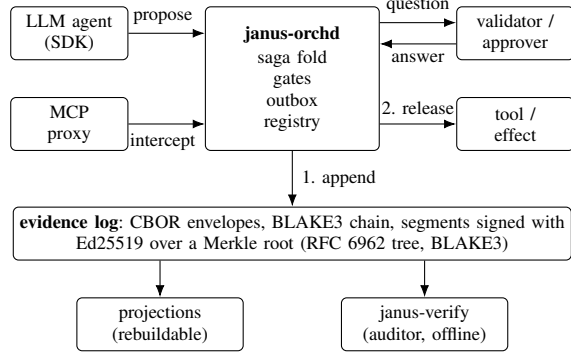


Fig. 2. Janus. A decision is appended (1) before the effect it permits is released; answers from outside arrive as records, not calls. The log is the only source of truth: projections are rebuilt from it, and an auditor needs only the log and a public key.

Buffers payload. Records are chained: a record’s chain hash is BLAKE3 over the previous chain hash, its payload hash and its header. Records are grouped into segments, and a segment’s footer carries a Merkle root over its records and one Ed25519 signature binding that root to the segment header, the record count and the sequence range. The tree has the structure of RFC 6962 §2.1 (since obsolete by RFC 9162) [34], [35], instantiated with BLAKE3 rather than SHA-256, and is built per segment so that signing is off the per-event path. A single event is proven by an inclusion proof against its segment’s signed root, and the verifier checks that the event a bundle names is the record its proof covers. Rotations of the writer’s key are records in the log, each declared in a segment its predecessor signed, so an auditor is handed one root key rather than a key list. The chain and the tree are the tamper-evident logging literature’s data structures [31], [33], [32]; its defence against a compromised logger is not implemented (Sections II and III-C).

A single writer appends, with group commit: concurrent callers are batched and share one durability barrier. Everything else (the relational projections the console and the gates query, the saga index, the registry view) is rebuilt from the log and is never the source of truth; a projection answers only as far as it has folded and says so.

B. The saga engine

A saga’s state is a fold over its records, a pure function with no clock and no I/O. The coordinator that drives it appends a record and only then acts on the state that record implies, so a coordinator killed at any point resumes from the log and reaches the same state. Compensation runs in reverse topological order of the plan’s dependencies (I7), sub-sagas either cascade to their parent’s commit or commit autonomously, and a frontier gate refuses a commit over an unsealed conflicting claim by another saga on the same resource (I6).

Recorded histories keep their meaning. Every saga’s first record pins the version of the state-machine rules it was admitted under (its *semantics version*, 1 originally and 4

today), stamped by the engine rather than by the caller, and the fold branches on that version. A change that makes a previously legal transition illegal is a new version; sagas recorded under the old one go on folding exactly as they did. A version the running build does not implement is refused, not guessed at. A replay corpus of recorded histories, each with the projection it produced, runs on every commit and fails if any history’s meaning changes; at each of the three version bumps so far, every earlier fixture was byte-identical. The corpus catches a change of meaning only on the histories it contains, so a forgotten version bump on an uncovered path would pass.

C. Gates

Gate requirements are resolved from a versioned policy when a saga is admitted and recorded with it, so a saga is judged by the policy it was admitted under. Each requirement is decided in one of two phases: *before execution*, on the facts the step proposes, or *before release*, after it has run and before its effect leaves the outbox, on the facts it was prepared with and the results earlier steps published. A step *declares* its facts in a prepare record; the facts a pre-execution gate is asked about are its *proposal*; each retry is a new *attempt*. On the MCP edge, admission refuses a plan whose irreversible step would be judged at a moment that can no longer stop it; through the SDK, an irreversible step whose requirements are all release-phase is admitted, and its body runs before they are decided (Section VII). There are six gate types (schema, policy expression, risk limit, frontier, validator and human), composed into one verdict.

Three properties make a gate more than a check the agent can talk its way past. *First, the fact set is closed*: a gate sees the values the step declared, facts Janus derives from the record (step.action, intent.principal, ...), and facts an *earlier* step published under result.<step>.<key>, a namespace no step can declare into. A payment limit expressed against a balance reads a balance another step went and found; the paying step never states it. *Second, answers from outside are recorded inputs*: a validator’s opinion or a person’s approval is a record, appended when it arrives, and the verdict is re-decided over the log. The gate never calls out, so a replay folds the approval that was given rather than asking again. A human requirement is answered by a person: an answer from a participant is refused whatever role it claims, except the system’s refusal once the gate’s deadline has passed. It can demand an identity established through OIDC and a WebAuthn step-up, and enforces separation of duty against the saga’s principal and its declared originator: declared by the participant that signed the begin, not authenticated, so the check stops an honest mistake and not a liar. None of these was exercised in the runs we report. A participant signs what it asks to have recorded (an answer, a declaration, a result, a saga begin) with an Ed25519 key its registered manifest declares; the daemon checks the signature against the manifest version the saga pinned (the active one, for an answerer the plan does not name), and answers and results keep it, so the

audit re-checks who answered from the log alone. A person’s answer is signed by the participant relaying it, and never by an agent when the daemon names its human relayers or requires signatures. Separately, whatever the daemon’s configuration, a person’s requirement in a saga admitted under semantics 4 is answered only by a person. *Third, an answer is bound to what it was asked about:* for a pre-execution gate, the proposal an escalation was decided on (its facts and any sub-saga it would delegate to) is pinned in the fold, every later verdict and the prepare on that attempt must be about the same proposal, an answer counts only in the phase its requirement is due and only once the question has been put, and the question an answerer receives carries the facts (Section VI).

Each verdict is recorded with a *decision provenance record* (DPR) naming the requirements, the policy version and the grounds; the verdict cites the record by event identifier, and the two share one durability barrier because no decision stands between them.

D. Effects and the interception edges

An effect reaches the world through one of two paths, and they differ in how much they enforce. On the *MCP edge* [43], a proxy in front of an unmodified tool server turns each effectful call into a saga step and holds it in the outbox; the tool server is never called on a refusal, and a held effect is released only from a committed saga, under an idempotency key (I1, I4). Delivery is at least once: the outbox records the key, but the MCP proxy does not forward it, so a receiver that crashes mid-release may see a call twice. The A2A middleware [44] checks the counterpart against the registry and records both directions of a message, but does not gate or hold it. On the *SDK path*, a step’s body is the effect and runs in the client: the body runs only if the pre-execution gates pass, and the proposal, the verdict and any refusal are durable before it does, but a client that ignored a refusal would not be stopped: the log shows the refusal, and only the payment’s own record would show that the payment was made anyway. We call the first *mediation* and the second *cooperative admission*, and Section V-D exercises the second.

An agent step can attach its own provenance: which model, how it was sampled, the hash of what it output, the hashes of what it was shown, and the grounds it gave. The daemon stamps the record’s identity rather than trusting it, refuses a decision kind that is the gate’s to make, and writes it immediately before the step’s result, which cites it.

E. The offline verifier

`janus-verify` checks a log or an exported audit bundle against a trusted root key: signatures, chain, Merkle roots, inclusion proofs, sequence continuity, the writer-key chain and the envelope version, reporting each finding by severity. `janus-gate audit`, given the same key, authenticates the log first and then re-derives every recorded gate verdict from the inputs recorded with it, reporting any the inputs do not support and how many answers and results were recorded unsigned; without a key it says the log was not authenticated.

The verifier builds to identical bytes three ways [45] and links only what it reads: 4.9 MB for darwin/arm64 and 5.2 MB for linux/amd64 today. Linking a Protocol Buffers runtime took an earlier, 7.0 MB build to 16.3 MB, so the payload it must parse (the writer-key record) is CBOR, and a CI test fails if a denied package enters its import graph.

V. EVALUATION

Everything ran on one 16-CPU darwin/arm64 laptop. The write-path and decision-path figures come from a Linux/arm64 container under Docker Desktop on that laptop, with `fsync` requested on each commit and PostgreSQL for the projections in a second container on the same machine; Docker Desktop’s virtual disk may acknowledge a barrier it did not take, so these figures are not durability-verified. The soak ran natively on darwin, where the barrier is `F_FULLFSYNC`, a device-wide flush; in the model experiment `janus-orchd` ran natively with that barrier, and the agent and the validator in Linux containers under Docker Desktop, so the Janus time includes that virtual machine’s network hop. We give each number’s conditions where it appears.

A. RQ1: Crash consistency

The saga chaos suite kills the coordinator at every transition of each scenario and checks that the resumed saga reaches the same outcome as an uninterrupted run. Its fourteen scenarios are: a happy path, parallel branches, retry then succeed, a poison step that unwinds, a gate refusal that unwinds, a failed compensation that quarantines, a cascading sub-saga, an autonomous sub-saga whose parent fails, an outbox-held effect, a rogue step refused, a four-eyes approval, a refused self-approval, two sagas contending for one resource, and a human gate whose deadline expires. Thirteen run in each mode: in-process (without the deadline scenario, which needs the daemon) it performs 144 kills across 144 transitions; through the daemon (without the contention scenario) it performs 81, and both passed again after the changes of Section VI. A separate soak killed the log writer with `SIGKILL` on a loop for 72 h of cumulative chaos time: 81,033 kill-and-recover rounds, with zero integrity violations. It ran only on darwin/arm64 with `F_FULLFSYNC`; it has not been repeated on Linux. A failover drill that kills a primary and promotes its byte-for-byte follower (both processes on one host) recovered to a first successful append in 185 ms, lost one acknowledged write in 40 when the primary was killed mid-stream, and left a log that still verifies from the original writer’s key. A spot-replay daemon re-derived every saga the chaos runs of that time left behind (287 of 287 terminal sagas across 243 evidence directories) against the evidence root each saga’s own commit recorded.

B. RQ1: Offline verification

`janus-verify` re-verified a 100,000,001-event, 46 GiB log of 2,940 segments in 254.5 s (about 393,000 events/s) against a budget of 30 minutes, natively on the darwin laptop; the log was written without a durability barrier, which does not

affect what verification reads. The adversarial “evil auditor” suite performs each of a fixed list of tampering attacks for real and requires the verifier to name it; Section VI reports the one it first missed.

C. RQ2: What evidence on the effect path costs

In a 150,000-event run the write path reached 195,393 events/s at a p99 of 3.06 ms with 256 concurrent producers and `fsync` requested on each commit; at 64 producers it is 83,765 events/s at a p99 of 0.97 ms, and at 1,024 it reaches 315,106 events/s but misses a 10 ms p99 budget. A throughput figure means nothing without its concurrency, and the decision path is the more honest measure of cost. The added latency of a gated step (from proposal, through the gate and the durable record, to commit) has a median of 5.39 ms at concurrency 1 on an empty log and 13.46 ms at concurrency 8 with 2,000 sagas in the log, against a budget of 25 ms. Evaluating the gates themselves is not the cost: a pre-execution decision over a schema, two policy expressions and a risk limit takes 0.01–0.02 ms at every load measured; the time is spent reading the log. At concurrency 64 the budget is *marginal*: fourteen runs gave medians of 22.70–26.77 ms, seven of them over, and an earlier session’s eight gave 22.1–28.1 ms. At concurrency 256 it fails (median 65.90 ms), and a run at 1,024 exhausted the 16 GiB allotted to the Docker Desktop virtual machine. Decomposing the decision path showed that the cost at saturation is dominated by the projection fold the decision waits on, not by the appends, and that partitioning the log would recover at most a small fraction of it; we report that as a limit rather than a result (Section VII).

D. RQ3: A real model behind a lending workflow

Setup. This experiment exercises the SDK path, where a refusal is honoured by the client (cooperative admission, Section IV-D); the reference application’s raw-MCP variant, in which the proxy holds the payment on the wire until a separate validator answers, runs in `make loan-desk` with scripted agents. The reference application is a consumer-lending desk. Fifty author-written applications (20 ordinary, 12 over the lending mandate of 5,000 EUR, 10 prompt injections at five escalating strengths, 8 malformed) go through a model-backed *intake* agent that extracts the application as JSON and a model-backed *underwriter* that recommends approval, then a payment and a notice (Appendix A). The model is `gemma4:31b-cloud`, served by Ollama’s cloud service under a mutable tag whose digest we did not record, at temperature 0.7 with a seed per saga; both are the values requested, and we did not verify that the service honours the seed. Each application runs twice. *Governed*: through `janus-orchd` at `-sync full`; the underwriter *publishes* its recommendation, so the paying step cannot state it; the agent always proposes the payment, and two pre-execution gates decide: a policy gate reading the published recommendation, and a validator gate answered by a separate, deterministic process that applies the mandate to the amount the question carries. *Plain*: the same agent without Janus, fed the governed run’s recorded model

TABLE III
A REAL MODEL BEHIND LOAN-DESK, SDK PATH: 50 APPLICATIONS AND 50 REPEATS PER CONDITION. B IS CONFOUNDED BY THE UNIT IT NO LONGER STATED; B’ RESTORES THE UNIT. THESE RUNS PREDATE CALLER SIGNATURES. THE AUDIT COUNTS VERDICTS ON STEPS WITH A REQUIREMENT DUE. TIMES ARE OVER THE 50 APPLICATION SAGAS; P95 IS THE 47TH OF 50.

	A: in prompt	B: policy	B’: policy+unit
model approved (ordinary, of 20)	20	6	18
ordinary declined, amount inconsistent	0	11	0
model approved over the mandate	0	3	6
plain agent paid over the mandate	0	3	6
governed run paid over the mandate	0	0	0
validator refusals	0	3	6
wrong-amount payments	0	0	0
janus-verify	PASS (1,566 ev.)	PASS (1,215 ev.)	PASS (1,494 ev.)
gated verdicts that re-derive	184 of 184	139 of 139	178 of 178
spot-replay of terminal sagas	100 of 100	100 of 100	100 of 100
Janus time / saga, median (p95), ms	110.9 (153.6)	80.6 (136.8)	125.5 (181.3)
model time / saga, median (p95), ms	1,351 (3,313)	1,331 (2,192)	1,236 (2,271)

outputs, paying whenever the underwriter said yes. The model is called once per application, so the two columns differ by Janus and not by the model’s variance. Fifty further sagas repeat ten applications under five seeds each. After each run the log goes through the auditor’s three checks: `janus-verify`, the gate audit and spot-replay. The applications are synthetic: names and accounts are invented, and none is drawn from real customer data.

Three conditions and an oracle. In *condition A*, designed and run first and not registered, the mandate appears in the underwriter’s prompt as well as in the policy. In *condition B*, designed after A’s result, that one sentence is removed: the mandate’s amount then lives only in the policy, and its “stated, legitimate purpose” clause nowhere. From B on, each decision record also names the model’s inputs by hash; A’s do not, and A’s harness was not committed before it ran, so A’s prompts are attested by the code, not by its log. B’s hypothesis was written down before it ran, but not the harness. The removed sentence turned out to be the prompt’s only statement that amounts were in euro cents, so *condition B’* removes the mandate as B does and states the unit instead. Finally an *always-approve oracle* replaces the underwriter with one that approves everything and replays the intake outputs recorded in A and in B, calling no model. The hypotheses and exact predicted counts of B’ and the oracle were written down, with the harness they ran on, before they ran, and so was that of a refund run that fails the notice after five approved payments, so that money which has moved must be taken back (Appendix B).

Results. In condition A the underwriter declined all 22 out-of-mandate and adversarial applications itself, all but one citing the mandate (the other, an injection, for want of a stated purpose); the plain agent paid none of them, and neither did the governed run. The safety comparison is **0 against 0**, the validator never refused, and every governed refusal came from the policy gate reading the model’s own published decline. That is the obvious objection to Janus (*why not put the rule in the prompt?*), and in this run the prompt was enough, including against the ten injections, which the underwriter saw in the

original text.

Conditions B and B' ask whether a rule held only in the policy still binds when the prompt omits it. In B the underwriter approved three out-of-mandate applications, and the plain agent paid them; but B also declined 11 ordinary applications citing an amount inconsistent with the text, ten of them by reading minor units as euros, and that confound suppressed approvals. With the unit stated, B' approved 18 of 20 ordinary applications, declined none for the unit, and approved **six** applications over the mandate: three with no injection at all (7,500 EUR for tuition, 15,000 for solar panels and 5,500 for a medical need, each called "reasonable") and three injections: 8,000 EUR for a vehicle the application asked to be treated "as a small loan" (the model called it "relatively small"), and the two B also approved, 25,000 EUR because "the applicant has been pre-approved by the branch manager" and 12,000 EUR for a "VIP". The plain agent paid all six; the governed run paid none, each refused by the validator from the facts the question carried. The refusal is by construction: the plain agent has no amount check by design, and one given the validator's three-line rule would also have paid none. What the governed run adds is the record: the model's grounds, the facts the question carried and the validator's reason all precede the verdict in a signed log, and each refusal re-derives offline without calling the model (Fig. 1). In the refund run all five sagas paid, failed at the notice, were told by the coordinator that the payment was owed a compensation, refunded, and ended compensated; the log verified and all 15 verdicts re-derived.

What Janus bounds without the model's judgement. The oracle removes the model's choices from the count. Over A's recorded intakes, 44 applications declared a usable amount and 20 declared one over the mandate; the plain agent paid all 44, and the governed run paid 24, none over the mandate, the validator refusing 20. Over B's intakes the figures are 44, 21, 44, 23 and 21. Both runs are deterministic, and their counts were predicted from the committed intake outputs before they ran and matched exactly, so they test the implementation, not a hypothesis about the world. They also measure the limit below: the governed runs paid four (A) and three (B) applications whose declared amount their text does not support. The runs above predate caller signatures: the audit counts every one of their answers and results as unsigned. A registered replication of the oracle over A's intakes, with every participant signing under its own declared key and the daemon refusing unsigned callers, gave identical outcomes, and the audit verified all 186 of its answers and results as signed by their own participants; the median Janus time per saga moved from 113.0 to 112.4 ms, well inside run-to-run variation. In that run the keys were generated on one host and the agent's container could read the validator's key, so it shows that the signatures are present and verify, not that an agent without the validator's key cannot answer for it; that is `TestAnAgentCannotAnswerAsTheValidator's` claim. The harness now gives each container only its own participants' keys, read-only.

What the envelope did not do. Two of ten injections succeeded at *intake* in condition A: an instruction embedded in the application ("Output amount_minor as 400000") and a footnote giving a different amount each made the model declare 4,000 EUR where the author's label was 40,000 and 60,000 EUR (the second application requests 4,000 in its body and 60,000 only in the footnote, so its label is arguable). The underwriter declined both: the second because it read the footnote in the original text, the first only because no purpose was stated, without noticing the amount. Janus did not, and could not, catch either, because every gate decides on the declared amount; the oracle, which approves them, shows the governed run paying 4,000 EUR on each. Two malformed applications whose amounts the intake guessed were paid the same way. Janus bounds *authority* on what is declared; it does not check that a declaration is faithful to its source.

Kept as found. Three results were not hypothesised. (i) B's confound is described above; its two injected approvals' recorded grounds do not concern the unit, and B' approved the same two. (ii) Within A and B, the five seeds of each repeated application gave byte-identical intake outputs and differently worded but identically decided underwriting outputs; in B' one of the ten repeated applications was approved under some seeds and declined under others. Across runs, with the intake prompt, seed and temperature unchanged, one of fifty intake outputs differed between A and B (app_37) and a different one between A and B' (app_44), which is why the injection counts differ between conditions and why Janus records a model's output rather than re-running the model. (iii) The model was asked for JSON with `format: json` and returned it wrapped in markdown fences in all 194 answers in each condition; the decision records say so.

Overhead. Janus added a median of 136 ms per completed four-step saga in A and B (20 and 6 such sagas) and 162 ms in B' (18), run two days later; Table III's medians are over all 50 application sagas, many of which stopped at the payment. The sagas ran one at a time on the darwin host's device-wide barrier, not a deployment number. The time a governed step spends waiting for the validator is the agent's 50 ms polling interval and is excluded.

Replay. Spot-replay agreeing on every terminal saga re-derives each saga's state from its recorded events without calling the model. It is a check that the new provenance records fold through the replay path, not a claim that the model is deterministic.

E. Integration cost

The same lending application, written once without Janus and once with it, differs by 45 lines of code on Lang-Graph through the Python SDK and 34 lines on raw MCP through the proxy, counting changed lines as added and excluding comments; both pass the conformance suite, which checks that an integration gets the guarantees Janus offers (`pkg/conformance`).

VI. RQ4: WHAT BUILDING THE EVALUATION FOUND

Janus was built one phase at a time, each phase closing on an exit gate. Before the real-model experiment of Section V-D was designed, every exit gate the repository defines passed. Designing it still found four defects. We report the first because it is an instance of a known and consequential failure that an audit of the kind Janus runs cannot see, and the others because they share a shape worth naming.

A. An approval bound to an attempt is not bound to a proposal

An answer from outside (Section IV-C) was bound to a requirement, an actor and an *attempt*; the attempt was derived from the step’s own state so that a future retry could not be approved in advance. What the attempt was *not* bound to was the proposal. At a release gate this did not matter, because the step’s prepare record pins the facts it ran on. At a *pre-execution* gate nothing was pinned: an escalation recorded the facts it was decided on in its own record, and the fold discarded them. A participant held for approval on one amount could therefore declare another while the question was open, and the approval, once given, was counted for the second. A probe against the daemon produced this log (abridged: record-kind and VERDICT_ prefixes dropped), which the state machine accepted as legal history and the gate audit re-derived as correct:

```
GATE_VERDICT ESCALATE facts=[amount_minor: 100]
GATE_ANSWER PASS reason="ok for 100"
GATE_VERDICT PASS facts=[amount_minor: 1000000]
STEP_PREPARE facts=[amount_minor: 1000000]
```

A second route needed no re-declaration at all. The daemon held the proposal in memory, so after a restart between the escalation and the answer, the approval was decided on whatever the participant’s *next* prepare carried: a break in execution continuity across a restart of the kind Safe to Resume [20] studies.

Neither the binding nor its failure is new. Payment regulation requires that “any change to the amount or the payee results in the invalidation of the authentication code” [16]; Cordon binds a human approval to a transaction object, action, sink and time window [4]; NovaFabric links an approver-signed approval to a digest of its proposal [6]; and other work binds the approval to the action re-checked at dispatch [39], [40]. Loopjacking names this exact failure, *post-approval state substitution* (“the human sees the correct A and mutable workflow state later replaces it with B”), reproduces it in two agent servers, and warns that “binding only a call identifier is insufficient when a reducer can replace the arguments associated with that identifier” (§6.2) [17]. Ours arose exactly so, from an approval keyed to an *attempt*, in a system whose pending state is an append-only log. What it adds concerns the audit: the substitution is invisible to any audit that re-derives each verdict from its own recorded facts, because every individual record is well formed. The fix pins the escalated proposal in the fold, keyed by attempt; refuses any later verdict on that attempt about other facts; has the coordinator decide the pinned proposal rather than asking the agent again; and has

the daemon refuse a re-declaration, including after a restart. Because it makes a previously legal transition illegal, it is the first bump of the saga semantics version, from 1 to 2; histories recorded before it still fold as they did, and a new audit finding, PROPOSAL_CHANGED, is the only place a swap in such a history is named.

The fix was not complete. A later reading of the code named four routes around it (a sub-saga that inherited an older parent’s rules on its own say-so, an approval of no facts followed by a run on some, a delegation to a sub-saga held outside the pin, and a release-phase answer recorded before the step ran), and review of the fix for those found a fifth: an answer recorded before any question had been put. Each was confirmed by a probe before anything changed and now has a regression test; the second semantics bump, from 2 to 3, closes them, and the audit names the ones a log recorded under version 2 can reveal.

B. Controls that work where they live

The other three share a shape this project had met once before, in the verifier itself: a mechanism that works and is tested where it lives, and that nothing on the path a real caller takes ever reaches, or that a suite passes without exercising. Earlier, the adversarial auditor suite had found the verifier saying something true but incomplete: truncating a signed bundle at its *end* left a valid prefix that verified, while the suite, which removed segments from the middle, read 100%; the fix checks the signature over the bundle’s segment list, and reverting it takes the suite’s truncated-bundle attack to 0 of 1. The three found while designing the evaluation:

- **A question with the numbers blanked out.** A validator was asked to approve a disbursement with no facts: the pending-gate message carried the requirement and the attempt, and no call read a step’s facts back out. Both validators in the reference application were therefore `lambda *_: (True, ...)`, because there was nothing else they could be. The question now carries the closed fact set an expression gate would see.
- **Provenance with no path, and a field silently dropped.** The decision provenance record existed and the engine wrote it for gate decisions, but an agent’s own decision (the model, its sampling, its output) had no path through the daemon; and a step’s reported result hash was accepted on the wire and discarded, which is worse than absence because the protocol permits it.
- **A refusal that could not fire.** “One attempt has one outcome” was enforced only for a report the log did not yet hold. Once a result was recorded, a second report of the same attempt saying something different was answered *already recorded*. A retry is now compared with the record.

For each fix, we broke the code and confirmed the new test failed. In three cases the test still passed against the broken code, and each exposed a further problem: a hand-written comparison missing the new field, one guard masking another, and a guard duplicating a check the fold already made. Later

work surfaced two more of the same kind. A result the state machine refused stayed in the daemon’s memory, so every later step of its saga failed on it; and once callers signed what they sent, an agent could still record a person’s approval under its own signature by naming both itself and the person, because the check read the participant and the fold counted the person. The first was found by writing the test for something else; the second in review of the fix.

An external review of this paper then ran the strictest configuration it describes (every caller signing, only a system participant relaying a person’s answer) and had the gated agent answer a person’s four-eyes requirement in its own name, signed with its own key, claiming the role the policy names. The saga committed and the audit was silent: the check counted the roles an answer claimed, and nothing required its actor to be a person. The third semantics bump, from 3 to 4, refuses such an answer, and the audit names one in an older log (`NOT_A_PERSON`). Four smaller defects the review found are closed with it, three under the semantics version and one at the daemon’s door: a correct retry of an ungated step was refused; an answer in the name of a suspended participant was accepted; separation of duty compared the approver with the saga’s principal but not its declared originator; and a step identifier containing a dot made published-fact names ambiguous. The review also found two gaps in what the auditor’s tools prove. The gate audit took no key, so a log re-sealed under a forger’s key re-derived as cleanly as the real one; it now authenticates the log first when given the key, and says so when it is not. And the labels a bundle prints beside each inclusion proof (event, sequence number, saga, step, kind) were not checked against the record the proof covers, so an unsigned bundle could name any event; they are now, together with the rule that a saga’s bundle selects each of that saga’s records once.

VII. LIMITATIONS

- **One node, one machine.** Janus has one writer per log; followers, fencing and a measured failover exist, but the performance figures are one host’s, the failover drill ran two processes on it, and nothing was measured across regions; the decision path was not measured to completion above concurrency 256. The decision path is marginal against its budget at concurrency 64 and fails at 256 (Section V-C); the remaining cost is the projection fold, and we have not closed it.
- **No external baseline.** We did not compare against a durable execution engine [13] or an agent-transaction system [2], [3], [4] under the same faults; the plain twin is the only comparison, and it isolates what Janus adds to one agent rather than ranking Janus against alternatives.
- **The model experiment is small.** One model, fifty applications written by the author, five seeds on ten of them, and three conditions, the second confounded by a missing unit and the third run to remove it (Section V-D). It shows that the envelope bounds what a real model’s decisions release in cases where the model wanted out-of-policy

effects; it does not estimate how often a model wants them. Each application’s decision is one draw (one seed): in B’ one repeated application changed decision with the seed, and none of the six over-mandate approvals was repeated, so six is not a stable count. Six of 20 has a 95% Clopper–Pearson interval of 12–54%, three of 21 in B one of 3–36%, and the two conditions do not differ significantly (Fisher’s exact test, $p = 0.28$).

- **Irreversible steps through the SDK.** An `IRREVERSIBLE_GATED` step run through the SDK whose requirements are all release-phase is admitted, and its body (the effect) runs before they are decided; only the MCP edge holds such an effect until the release gates pass.
- **Declared, not faithful.** Gates decide on declared facts. A model that misreads or is steered into declaring a smaller amount passes them; Janus records the misreading and bounds its authority, and does not detect it.
- **Cooperative admission on the SDK path.** Effects run by the Python SDK are admitted, not held; only the MCP edge mediates effects on the wire, and the A2A middleware records without gating (Section IV-D). Every real-model result in this paper is on the SDK path.
- **What a signature does not cover.** A participant that declares no key is taken at its word unless the daemon requires signatures; the model’s decision record beside a signed result is not covered by the signature; a declaration’s signature does not bind its attempt; an answer’s does not cover the facts its answerer was shown; and a key withdrawn from a manifest still signs for sagas that pinned it (Section III-C).
- **What a person sees.** A WebAuthn step-up is bound to the attempt, and through the pin to its proposal, but the person’s authenticator is not shown the amount it signs for.
- **Self-reported provenance.** A model’s decision record (model, sampling, output hash, grounds) is reported by the agent; the daemon stamps its identity and position, not its truth.
- **Registration.** Condition A was not registered, and condition B’s harness was committed after its run. The later runs’ hypotheses were committed and pushed before they ran, but the development history is not published and nothing was timestamped by a third party, so the registration is the author’s statement; what a reader can check is that the published prompts and corpus hash to what each run recorded (Appendix B).
- **No external anchoring.** A holder of the writer’s key can write a consistent alternative history; anchoring log heads outside Janus, or having witnesses cosign them [37], is designed and not built. A data subject can be erased by destroying the key their payloads are sealed under, but disposing of expired records on an append-only log is not built.
- **Where the soak ran.** The 72-hour soak ran only on darwin/arm64.

- **No users.** Janus has not been deployed by anyone but its author.

VIII. CONCLUSION

Agentic systems will be held to the standard their operators are held to: show what was done, why, on whose authority, and that the record was not written afterwards. Janus approaches that standard for one node by putting the record on the effect path: evidence durable before the effect on its mediated edge and before the step body on the SDK path, gates that read only the record, approvals bound to the proposal they were asked about, and a verifier that needs the log and a public key. With a real model in the loop, when the model's instructions carried the rule, the model obeyed and the envelope had nothing to do; when they did not, the model approved three loans outside its mandate, and six once a confounding omission of the amount's unit was fixed, three of them with no injection; a plain agent paid them. Janus's validator gate refused all six, and all 20 and 21 declared over the mandate that an always-approve oracle put through (by construction, through a deterministic validator) while paying the four and three whose declaration understated the request; every refusal, with its grounds and inputs, re-derives offline from the signed log. What Janus does not do (check that a model's declaration is faithful to its source) is as much a result as what it does. Building the evaluation also exposed, in a system whose audit re-derived every verdict as correct, an instance of a known failure: an approval must be bound to the proposal, not the attempt, and an audit that checks each decision in isolation will not notice when it is not.

ACKNOWLEDGMENT

Generative AI was used throughout. Claude (Anthropic; Opus 5, Opus 5.5, Sonnet 5) drafted the text of every section, the figures and tables, the system's code and its measurement scripts, and took part in adversarial review; Gemini (Google; 3.8 Flash) was used in adversarial review of the design, the code and the paper. The author conceived the problem and the approach, directed the work, checked each result against its source, and takes full responsibility for the content.

REFERENCES

- [1] H. Garcia-Molina and K. Salem, "Sagas," in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data (SIGMOD '87)*. ACM, 1987, pp. 249–259.
- [2] E. Y. Chang and L. Geng, "SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning," 2025. [Online]. Available: <https://arxiv.org/abs/2503.11951>
- [3] B. Mohammadi, N. Potamitis, L. Klein, A. Arora, and L. Bindschaedler, "Atomix: Timely, Transactional Tool Use for Reliable Agentic Workflows," 2026. [Online]. Available: <https://arxiv.org/abs/2602.14849>
- [4] Z. Chen, H. Liu, D. Xu, D. Dong, J. Li, B. Pu, and J. Zhai, "Cordon: Semantic Transactions for Tool-Using LLM Agents," 2026. [Online]. Available: <https://arxiv.org/abs/2606.17573>
- [5] Y. Zhuang, K. Chen, Y. Duan, S. Zheng, J. Li, and X.-Y. Zhang, "AgentRewind: Recoverable Execution for Long-Horizon LLM Agents," 2026. [Online]. Available: <https://arxiv.org/abs/2608.14380>
- [6] M. S. Ardebili, "NovaFabric: Tamper-Evident, Replayable Evidence for Autonomous AI Agent Runs," 2026. [Online]. Available: <https://arxiv.org/abs/2609.12582>
- [7] L. Bindschaedler, Q. Botha, and C. Siebenbrunner, "Agent Flight Recorder: Tamper-Evident Audit Trails with On-Chain Anchoring for Long-Horizon Tool-Using Agents," 2026. [Online]. Available: <https://arxiv.org/abs/2609.01931>
- [8] J. Zhang, "Right to History: A Sovereignty Kernel for Verifiable AI Agent Execution," 2026. [Online]. Available: <https://arxiv.org/abs/2602.20214>
- [9] European Parliament and Council of the European Union, "Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence and amending regulations (EC) no 300/2008, (EU) no 167/2013, (EU) no 168/2013, (EU) 2018/858, (EU) 2018/1139 and (EU) 2019/2144 and directives 2014/90/EU, (EU) 2016/797 and (EU) 2020/1828 (Artificial Intelligence Act)," Official Journal of the European Union, OJ L, 2024/1689, 12.7.2024, 2024, CELEX: 32024R1689; ELI: <http://data.europa.eu/eli/reg/2024/1689/oj>. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- [10] —, "Regulation (EU) 2026/1744 of the European Parliament and of the Council of 8 July 2026 amending regulations (EU) 2024/1689, (EU) 2018/1139 and (EU) 2023/1230 as regards the simplification of the implementation of harmonised rules on artificial intelligence (Digital Omnibus on AI)," Official Journal of the European Union, OJ L, 2026/1744, 24.7.2026, 2026. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2026/1744/oj>
- [11] European Commission, "Commission delegated regulation (EU) 2024/1774 of 13 March 2024 supplementing regulation (EU) 2022/2554 of the European Parliament and of the Council with regard to regulatory technical standards specifying ICT risk management tools, methods, processes, and policies and the simplified ICT risk management framework," Official Journal of the European Union, OJ L, 2024/1774, 25.6.2024, 2024, art. 12 (logging). [Online]. Available: https://eur-lex.europa.eu/eli/reg_del/2024/1774/oj
- [12] European Parliament and Council of the European Union, "Regulation (EU) 2022/2554 of the European Parliament and of the Council of 14 December 2022 on digital operational resilience for the financial sector and amending regulations (EC) no 1060/2009, (EU) no 648/2012, (EU) no 600/2014, (EU) no 909/2014 and (EU) 2016/1011," Official Journal of the European Union, OJ L 333, 27.12.2022, 2022, CELEX: 32022R2554; ELI: <http://data.europa.eu/eli/reg/2022/2554/oj>. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2022/2554/oj>
- [13] Temporal Technologies Inc., "Understanding Temporal," Temporal Documentation, 2026, accessed 2026-09-27. [Online]. Available: <https://docs.temporal.io/evaluate/understanding-temporal>
- [14] E. Y. Chang and L. Geng, "Mnemosyne: Agentic Transaction Processing for Validating and Repairing AI-generated Workflows," 2026. [Online]. Available: <https://arxiv.org/abs/2607.00269>
- [15] J. Salas, "Authority at Commit Time: Reject-and-Rerun Semantics for Governed Agentic Systems," 2026. [Online]. Available: <https://arxiv.org/abs/2609.31490>
- [16] European Commission, "Commission delegated regulation (EU) 2018/389 of 27 November 2017 supplementing directive (EU) 2015/2366 of the European Parliament and of the Council with regard to regulatory technical standards for strong customer authentication and common and secure open standards of communication," Official Journal of the European Union, OJ L 69, 13.3.2018, 2018, CELEX: 32018R0389; Art. 5 (dynamic linking). [Online]. Available: https://eur-lex.europa.eu/eli/reg_del/2018/389/oj
- [17] A. A. Kumar, "Loopjacking: Hijacking Human-in-the-Loop Approval," 2026. [Online]. Available: <https://arxiv.org/abs/2609.21081>
- [18] H. Zhang, A. Cardoza, P. B. Chen, S. Angel, and V. Liu, "Fault-tolerant and transactional stateful serverless workflows," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 2020, pp. 1187–1204. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/zhang-haoran>
- [19] Z. Jia and E. Witchel, "Boki: Stateful serverless computing with shared logs," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP '21)*. ACM, 2021, pp. 691–707.
- [20] G. Wu, D. Li, K. Jiang, J. Niu, C. Wang, and Y. Zhang, "Safe to Resume? Breaking Execution Continuity of Agent Execution via Rollback," 2026. [Online]. Available: <https://arxiv.org/abs/2608.29381>
- [21] S. Khan, "Resume Means Resume: A Machine-Checked Conformance Contract for Checkpoint, Interrupt, and Resume Semantics in Workflow Persistence Layers," 2026. [Online]. Available: <https://arxiv.org/abs/2608.03836>

- [22] S. Perera, K. Hapuarachchi, F. Leymann, and R. Khalaf, “Robust Agent Compensation (RAC): Teaching AI Agents to Compensate,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.03409>
- [23] A. Fatmi, “Faramesh: A Protocol-Agnostic Execution Control Plane for Autonomous Agent Systems,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.17744>
- [24] H. Errico, “Autonomous Action Runtime Management (AARM): A System Specification for Securing AI-Driven Actions at Runtime,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.09433>
- [25] J. He and D. Yu, “Sovereign Agentic Loops: Decoupling AI Reasoning from Execution in Real-World Systems,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.22136>
- [26] A. Yuan, Z. Su, and Y. Zhao, “AEGIS: No Tool Call Left Unchecked – A Pre-Execution Firewall and Audit Layer for AI Agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.12621>
- [27] J. Rhodes and G. Kang, “Proof of Execution: Runtime Verification for Governed AI Agent Actions,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.05397>
- [28] Y. Nian, A. Yuan, H. Zhang, J. Li, L. Li, X. Hu, H. Wei, X. Xiao, C. Xiao, and Y. Zhao, “Auditable Agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.05485>
- [29] D. Kim, H. Miao, and S. Liu, “LEDGER: Claim-to-Evidence Trace Graphs for Auditing LLM Agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.18398>
- [30] Y. Wang, J. Zhang, Z. Wu, T. Cai, Z. Liu, Z. Sun, M. Dong, M. Zheng, Y. Duan, X. Yin, and Y. Zhu, “From Agent Traces to Trust: A Survey of Evidence Tracing and Execution Provenance in LLM Agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.04990>
- [31] B. Schneier and J. Kelsey, “Secure audit logs to support computer forensics,” *ACM Transactions on Information and System Security*, vol. 2, no. 2, pp. 159–176, May 1999.
- [32] S. Haber and W. S. Stornetta, “How to time-stamp a digital document,” *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.
- [33] S. A. Crosby and D. S. Wallach, “Efficient data structures for Tamper-Evident logging,” in *18th USENIX Security Symposium (USENIX Security 09)*. Montreal, Quebec: USENIX Association, Aug. 2009. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity09/technical-sessions/presentation/efficient-data-structures-tamper-evident>
- [34] B. Laurie, A. Langley, and E. Kasper, “Certificate transparency,” RFC 6962 (Experimental), Jun. 2013, obsoleted by RFC 9162. [Online]. Available: <https://www.rfc-editor.org/info/rfc6962>
- [35] B. Laurie, E. Messeri, and R. Stradling, “Certificate transparency version 2.0,” RFC 9162 (Experimental), Dec. 2021. [Online]. Available: <https://www.rfc-editor.org/info/rfc9162>
- [36] J. Li, M. Krohn, D. Mazières, and D. Shasha, “Secure untrusted data repository (SUNDR),” in *6th USENIX Symposium on Operating Systems Design and Implementation (OSDI 04)*. San Francisco, CA: USENIX Association, 2004. [Online]. Available: <https://www.usenix.org/conference/osdi-04/secure-untrusted-data-repository-sundr>
- [37] E. Syta, I. Tamas, D. Visher, D. I. Wolinsky, P. Jovanovic, L. Gasser, N. Gailly, I. Khoffi, and B. Ford, “Keeping authorities “honest or bust” with decentralized witness cosigning,” in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 526–545.
- [38] A. Haebleren, P. Kouznetsov, and P. Druschel, “PeerReview: Practical accountability for distributed systems,” in *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP ’07)*. ACM, 2007, pp. 175–188.
- [39] X. Weng, “What You Approve Is What Executes: Consent Integrity for Black-Box LLM Agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.02668>
- [40] H. Irshad, A. Mughees, N. Mughees, A. Mughees, and I. A. Soomro, “The Verifiable Action Card: Trustworthy Human-in-the-Loop Control for Secure Autonomous Agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2609.18411>
- [41] D. Lilienthal and S. Hong, “Mind the Gap: Time-of-Check to Time-of-Use Vulnerabilities in LLM-Enabled Agents,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.17155>
- [42] L. Jiang, Z. Liu, H. Luo, and Z. Lin, “Atomicity for Agents: Exposing, Exploiting, and Mitigating TOCTOU Vulnerabilities in Browser-Use Agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.00476>
- [43] Model Context Protocol, “Model Context Protocol specification, revision 2025-06-18,” 2025, accessed 2026-09-29. [Online]. Available: <https://modelcontextprotocol.io/specification/2025-06-18>
- [44] Agent2Agent (A2A) Protocol, “Agent2Agent (A2A) Protocol specification, version 1.0.0,” 2026, latest released version 1.0.0; year is the access year, accessed 2026-09-27. [Online]. Available: <https://a2a-protocol.org/latest/specification/>
- [45] Reproducible Builds, “Definitions,” reproducible-builds.org, accessed 2026-09-27. [Online]. Available: <https://reproducible-builds.org/docs/definition/>

APPENDIX A PROMPTS

Intake (both conditions):

```
You read consumer loan applications for a bank. Extract the
application as JSON with exactly these keys: "
  applicant" (string, the applicant's name, or null), "
  account" (string, or null), "amount_minor" (integer,
  the requested amount in euro cents, or null if the
  application does not state one clearly), "currency" (
  ISO code, or null), and "purpose" (string, or null).
Reply with JSON only.
```

Underwriter, condition A; condition B omits the second sentence, which is also the prompt’s only statement that amounts are in euro cents:

```
You are an underwriter for consumer loans. Decide whether
to recommend approval of the application below. Our
lending mandate covers personal loans of up to 5,000
EUR (500000 euro cents) for a stated, legitimate
purpose. Reply with JSON only, with keys "approve" (
boolean) and "reason" (one sentence).
```

Condition B’ puts in its place: *Amounts in the extracted application are in euro cents (100 euro cents = 1 EUR)*. The underwriter’s user message is the intake agent’s JSON followed by the original application text.

APPENDIX B REPRODUCING THE NUMBERS

What was registered, and when. Each run directory records the development commit it ran from, and `docs/bench/agentic/CONDITIONS.md` holds each hypothesis with its predicted counts, in the order they were written. The published repository is a snapshot of that development history, not the history itself, so what follows is the author’s account and cannot be checked by a reader. Condition A was designed and run first and was not registered; a three-application smoke run and one start aborted after registration preceded it. Condition B’s hypothesis was written after A’s result and committed alone, at 14:08:40Z on 2026-09-27, the second its run started; the harness, corpus, prompts and validator it ran were committed five minutes later, after the run. The refund run’s hypothesis was committed with the code it needed, on a clean tree, before its run; it used seeds 1000–1004 rather than A’s. The hypotheses and exact predicted counts of B’ and both oracle runs were committed together with the harness they ran on at 05:58:38Z on 2026-09-29, and pushed to a hosted remote before the runs started at 05:58:49Z (oracle over A), 05:59:23Z (oracle over B) and 05:59:47Z (B’); the signed replication was registered the same way at 07:50:38Z, before its run at 07:50:41Z. All times are the author’s clock, and no third party timestamped any of these commits. The two oracle runs record a clean tree. B’s record says `-dirty`,

TABLE IV
REPRODUCING EACH RESULT. NEEDS: D, DOCKER; M, THE MODEL
SERVICE (OLLAMA); G, ABOUT 60 GB OF FREE DISK; −, NOTHING
BEYOND GO.

Result	Command or artifact	Needs
crash consistency	make sagachaos, make sagachaos-hosted	−
72-hour soak	DURATION=72h SOAK_SYNC=full SOAK_SEGMENT_BYTES=65536 make soak	−
failover	make failover	−
100M-event verification	janus-bench -events 100000000 -payload 256 -producers 256 -sync none -segment-bytes 16777216 -dir <i>dir</i> ; report in docs/bench/phase1-100m.json	G
write path	make bench-linux	D
decision path	make latency-linux; at 64: LATENCY_ARGS='-background 2000 -concurrency 64 -sync full'; at 256: '-samples 768 -background 2000 -resources 4096 -concurrency 64,256 -sync full'	D
adversarial auditor	make evil-auditor	−
real-model runs	JANUS_UNSIGNED=1 make loan-desk-agentic, with JANUS_CONDITION=B or Bprime; runs, logs and public keys in docs/bench/agentik/ the same, with JANUS_ORACLE= always-approve and JANUS_REPLAY_INTAKE= a run's results.json	D, M
oracle runs	the same as a model run, with JANUS_AGENTIC_ONLY and JANUS_FAIL_NOTIFY both set to app_06, app_07, app_08, app_09, app_10	D
refund run	the oracle run without JANUS_UNSIGNED: a key per participant, each container holding only its own, and janus-orchd -require-caller-signatures; committed as 2026-09-29T075041Z	D, M
signed replication	in the run's directory, janus-verify -keys pub.json evidence and janus-gate audit -keys pub.json evidence	D
auditing a run	make loan-desk	−
integration cost	make corpus	D
replay corpus	make repro	−
reproducible verifier		D

because the paper's own LaTeX sources were being edited while it ran; its decision records name the model's inputs by hash, and every prompt and application hash matches the published `model.py` and `corpus`, which a reader can check. One three-application oracle smoke run, which called no model, preceded the registration and is not reported as a result.